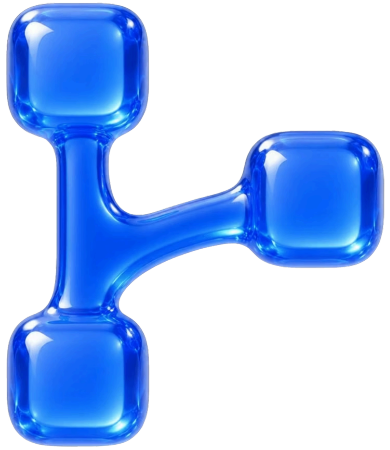
GIT FUNDAMENTALS

From Zero to Professional Workflow



تعلّم كيف تحفظ نسخ مشروعك البرمجي وتديره بطريقة منظمة

المدرب: عبدالحكيم قنبر · bremix technolojy · bremix.tech · 60 Minutes

PRACTICAL WORKSHOP VERSION CONTROL

Made with GAMMA

ماذا ستتعلم؟ ولماذا نحتاج GIT؟

الأهداف، ومقارنة الطريقة القديمة بالطريقة الجديدة

WITHOUT GIT · الفوضى اليدوية	WITH GIT · الاحترافية والأمان
نسخ كثيرة من الملف نفسه: `final-v2` و `final-final`.	**سجل كامل:** لقطة محفوظة (Commit) لكل خطوة.
تضيع تعديلاتك إذا كتبت فوق ملف زميلك.	**فروع منفصلة:** تطوّر ميزة جديدة دون أن تعبث بالكود الأساسي.
لا تعرف: *من كتب هذا السطر؟ ومتى؟ ولماذا؟*	**تعاون آمن:** تراجع الكود وتدمجه قبل رفعه على GitHub.
الرجوع لنسخة قديمة يحتاج ساعات من البحث اليدوي.	**ثقة تامة:** ترجع لأي نقطة سابقة بأمر واحد.

ROADMAP: INIT → STAGE → COMMIT → BRANCH → MERGE → GITHUB

Made with GAMMA

الفرق والمصطلحات: GITHUB ليس GIT

الفرق بين البرنامج على جهازك والموقع على الإنترنت

وجه المقارنة	Git (المحرك المحلي)	GitHub (المنصة السحابية)
النوع والمفهوم	برنامج لحفظ نسخ المشروع، ويعمل على جهازك.	موقع على الإنترنت يحفظ مستودعات Git ويشارك فيها الناس.
الاتصال بالإنترنت	يعمل بدون إنترنت تماماً.	يحتاج إنترنتاً لرفع النسخ ومزامنتها (Remote).
واجهة الاستخدام	أوامر في Terminal، أو أدوات بالنوافذ.	صفحة ويب يدير منها الفريق العمل ويراجع الكود.
ميزات التعاون	يصنع ملفات التغييرات ويدمجها على جهازك.	Pull Requests، Code Reviews، Issues، و CI/CD Actions.

Repository: مجلد المشروع مع سجله

Commit: لقطة محفوظة لحالة المشروع

Branch: نسخة جانبية تعمل فيها بحرية

Remote: النسخة المرفوعة على الإنترنت

مصطلحات أخرى: Merge · Clone · Push · Pull

Made with GAMMA

المناطق الثلاث ومسار العمل اليومي

كيف تنتقل ملفاتك من جهازك إلى السحابة، خطوة بخطوة

1. WORKING DIRECTORY

مجلد العمل: هنا تكتب الأكواد وتحفظ ملفاتك.

2. STAGING AREA

منطقة التجهيز: تختار التغييرات التي تريد حفظها بـ `git add`.

3. LOCAL REPOSITORY

المستودع المحلي: تحفظ لقطة ثابتة داخل `.git` بـ `git commit`.

4. REMOTE (GITHUB)

المستودع البعيد: ترفع عملك على الإنترنت ليراه الفريق بـ `git push`.

الدورة اليومية:

```
git status ← git add .
git commit -m "Meaningful commit message"
git push
```

الرفع لا يتم تلقائياً؛ عليك ضبط remote أولاً.

Made with GAMMA

تثبيت GIT وأول مستودع

حمّل Git من الموقع الرسمي git-scm.com/downloads، ثم افتح الـ Terminal.

CONFIG · الضبط لأول مرة

تأكد أن Git مثبت، ثم اكتب اسمك وبريدك. سيظهران مع كل Commit.

```
git --version
"git config --global user.name "Your Name
"git config --global user.email "your@email.com
```

استخدم نفس البريد الذي في حساب GitHub.

INIT · إنشاء المستودع الأول

أنشئ مجلداً، ثم حوّله إلى مستودع، وتحقق من حالته:

```
mkdir my-first-project
cd my-first-project
git init
git status
```

يُنشئ Git مجلداً مخفياً اسمه .git يحفظ كل سجل المشروع.

تنبيه: لا تحذف المجلد .git.

Made with GAMMA

STATUS و ADD و COMMIT

GIT STATUS

يُظهر حالة ملفاتك:

- **Untracked:** ملفات جديدة لم يتابعها Git بعد.
- **Modified:** ملفات تغيّرت ولم تجهّزها بعد.
- **Staged:** ملفات جاهزة للحفظ.
- **Clean working tree:** لا توجد تغييرات جديدة.

GIT ADD و GIT COMMIT

`git add .` أو `git add README.md` يضع التغييرات في قائمة الانتظار، ولم تُحفظ بعد.

`git commit -m "Initial commit"` يحفظ لقطة دائمة في السجل؛ مثل `git commit -m "Add hero section"`.

الـ Hash هو رقم تعريف النسخة. استخدم `git log --oneline` لعرض السجل، و`git diff` لمراجعة التغييرات.

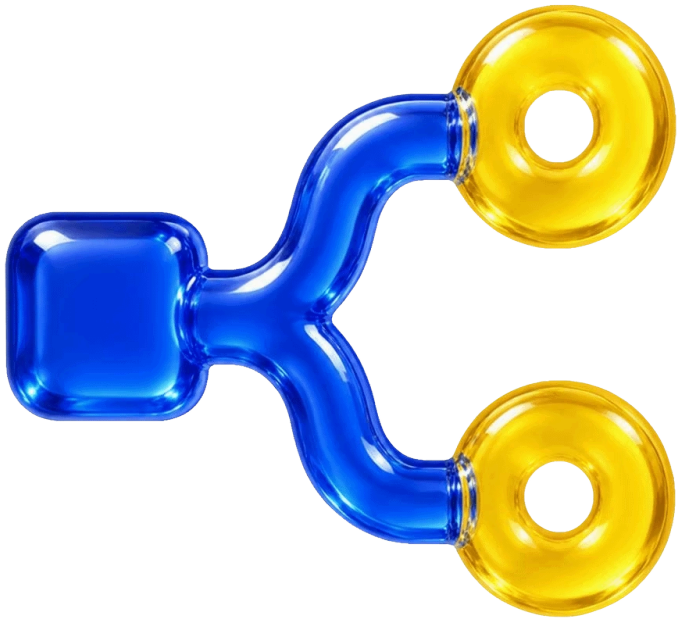
رسائل جيدة: `Add login page` · `Fix navbar responsive issue`

رسائل سيئة: `asdasd` · `Test` · `Changes` · `Update`

Made with GAMMA

الفروع BRANCHES

اعمل على ميزة جديدة في نسخة منفصلة، دون أن تُفسد الكود الأساسي الذي يعمل.



BRANCHING STRATEGY · استراتيجية التفريع

main هو الكود الأساسي الذي يعمل. ومنه تتفرع فروع جديدة مثل feature للميزات و bugfix للإصلاحات. بعض المشاريع القديمة تستخدم master.

```
git branch
git switch -c feature-homepage
git branch
git checkout -b feature-homepage
git switch main
```

git branch يعرض الفروع، والفرع الذي عليه علامة (*) هو الذي تعمل عليه. git switch -c feature-homepage ينشئ فرعاً جديداً وينقلك إليه. الأمر القديم checkout -b الآن. أمر git switch أسهل للفهم.

Made with GAMMA

الدمج MERGE وحل التعارضات CONFLICTS

MERGING · خطوات الدمج الطبيعي

بعد تجربة الميزة، ارجع إلى main ثم ادمج فرعك فيه:

```
git switch main
git merge feature-homepage
```

قبل الدمج: الفرعان منفصلان. بعد الدمج: كل التغييرات صارت في main.

MERGE CONFLICT · فض النزاع

يحدث التعارض حين يغيّر فرعان السطر نفسه، ولا يعرف Git أيهما يختار:

```
<<<<<<< HEAD
<h1>أهلاً بكم في متجرنا</h1>
=======
<h1>مرحباً بكم في منصتنا</h1>
>>>>>>> feature-homepage
```

الحل: افتح الملف واختر النص الصحيح، واحذف علامات `<<<` و`===` و`>>>`. ثم نفذ `git add .` وأكمل الدمج بـ `git commit` إذا طلب Git.

قاعدة ذهبية: لا تخف من التعارض. Git يحمي كودك ولا يحذف شيئاً دون قصد.

Made with GAMMA

GITHUB والتعاون

ربط المستودع المحلي بالسحابة وسير عمل الفريق

CONNECTING · ربط المستودع

أنشئ مستودعاً فارغاً على GitHub، ثم صِله بجهازك وارفع الكود إليه:

```
git remote add origin
https://github.com/USERNAME/REPOSITORY.git
git branch -M main
git push -u origin main
```

`-u` التتبع للرفع الأول ؛ يفعّل remote اسم الـ **origin**.

يمكنك الدخول عبر المتصفح أو Personal Access Token أو SSH. لا تكتب كلمة المرور داخل الأوامر.

COLLABORATION · دورة عمل الفريق

DEVELOPER A ← GITHUB → DEVELOPER B

تنزيل المشروع كاملاً إلى جهازك.	`git clone`
يجلب التحديثات دون أن يغيّر ملفاتك.	`git fetch`
يجلب ويدمج.	`git pull`
يرفع التغييرات.	`git push`

بعدها يستخدم الفريق **Pull Requests** و**Code Reviews** لمراجعة الكود قبل دمجه في `main`.

Made with GAMMA

المختبر العملي، الاختبار، والختام

HANDS-ON LAB · تمرين مشروع كامل

```
git init
echo "My First Website" > index.html
"git add . && git commit -m "Add homepage
git switch -c feature-about
echo "About Bremix" >> index.html
"git commit -am "Add about section
git switch main && git merge feature-about
git log --oneline --graph --all
```

شجرة السجل: الأمر الأخير يرسم لك الفروع وكيف اندمجت ببعضها.

QUICK QUIZ · اختبار السرعة

1. ما الفرق بين Git و GitHub؟
2. ما دور كل من `git add` و `git commit`؟
3. لماذا نستخدم Branches؟
4. ما الفرق بين `git push` و `git pull`؟

شكراً لحضوركم — Thank You!

عبدالحكيم قنبر · bremix.tech · bremix technolojy

Made with GAMMA